Where No One Has Gone Before

A Mars rover adventure in Three.js

Technical Presentation and User Manual

Interactive Graphics course - Prof. Marco Schaerf
Sapienza University of Rome
Author: Giulia Grossi - Grossi.1979491@studenti.uniroma1.it

July 2026 — project “GGame”

Live demo (GitHub Pages): see the link in the repository README

1. Introduction

Where No One Has Gone Before is a single-player exploration game set on Mars, written from scratch in JavaScript on top of Three.js. The player drives a small tracked rover across an endless procedural landscape of dunes, craters and boulders, under a night sky with recognizable two constellations. The mission has two goals: collect a set number of green crystals, which are hidden inside rare gold-veined purple boulders and must be freed with the rover's laser, and carry a set number of ordinary red boulders to the cargo pad of a blue rocket parked near the landing site. When both goals are complete a countdown starts and the rocket lifts off; ten seconds into the climb "MISSION ACCOMPLISHED" appears written across the night sky as a new constellation of yellow stars. A five-minute timer, armed by the first rock loaded onto the pad, keeps the pressure on.

The project was designed around the course requirements: the rover is a deep hierarchical model whose animations exploit its structure; the scene combines four light sources with six procedurally generated texture maps of four different kinds (color, normal, bump and metalness); every animation, including all the physics, is implemented by hand in JavaScript; and the game offers a wide range of user interactions, from driving and camera switching to a difficulty selector. No models, animations or physics engines were imported. This document is both the technical presentation and the user manual: section 3 addresses the player, sections 4 to 6 the reader interested in how things work.

2. Environment, libraries and assets

The project uses **Three.js r128** as its rendering library, included as a local minified copy so that the repository is fully self-contained and runs offline. Everything else is plain browser technology: one HTML page with a CSS overlay for menus and HUD, and a single JavaScript file, `GGame.js`, of roughly 1,900 commented lines that contains the whole game. There is no build step, no bundler and no package manager: opening the page in a browser is the entire deployment.

Not developed by me: only `three.min.js` (the rendering library) and `soundtrack.mp3` (the background music track). No 3D models were downloaded or created in external modelers: every object like rover, rocket, terrain, rocks, sky is built in code from Three.js primitive geometries (boxes, cylinders, spheres, cones, dodecahedra, planes and point clouds). No animation was imported and no physics engine is used: the ballistic arcs, rolling boulders and collisions described in section 6 are hand-written integrations, small enough to run at full frame rate.

Repository layout: `index.html` (page, styles, HUD and menu markup), `GGame.js` (all the code), `three.min.js`, `soundtrack.mp3`, this document and a small presentation.

3. User manual

3.1 Start screen and game modes

The game opens on a start screen drawn over the live Mars scene: the title, two difficulty buttons, a play button and a music toggle. The selected difficulty glows; EASY is preselected. Pressing the play glyph removes the overlay and unlocks the keyboard. Music can be started right from the menu, because a button click is the user gesture browsers require before playing audio; it can be toggled at any time later with the M key. Changing difficulty rewrites the goals and every goal number printed in the on-screen hints.

	EASY	HARD
Crystals to collect	10	17
Red rocks to load	5	12
Mission timer	5 minutes	5 minutes

3.2 Controls

Key	Action
◂ ▲ ▼ ▸	Drive: accelerate, brake/reverse, steer
`1 / 2`	Tilt the head (and both camera eyes) up / down
`9 / 0`	Pan the head left / right
`P`	Stretch the arms and grab the red boulder in front of the rover
`O`	Set the carried boulder down just beyond the front (on the cargo pad: load it into the rocket)
`T`	Throw the carried boulder forward on a ballistic arc
`F`	Fire the green laser at the purple boulders (disabled while carrying a rock)
`SPACE`	Fire the jump rockets: a ballistic hop under Mars gravity (disabled while carrying a rock; no double jump)
`C`	Toggle between the chase camera and the first-person eye camera
`L`	Toggle the two headlight spotlights
`M`	Toggle the soundtrack
`R`	Reset the rover pose to the landing site (mission progress is kept)
`V`	Skip straight to the finale the 3·2·1 countdown, lift-off and the victory constellation (demo shortcut)

3.3 Head-up display

The top-left corner shows the rover's map coordinates and, in eye view, a reminder of the active camera. The bottom-left corner holds a dashed reference card with all the controls, in the same style as the start screen. Two counters live in the top-right corner and appear only when they become relevant: a green crystal counter after the first pickup and an orange cargo counter after the first rock is loaded, each shown as "n / goal" with a check mark on completion. The mission timer appears at the top centre when armed and turns red in the last thirty seconds. Just below it sits the rocket compass: a small arrow over the distance in metres that always points toward the launch pad. It is visible from the moment PLAY is pressed, rotates against the rover's heading so that "up" always means "straight ahead", dims once the rover is within a few metres of the pad, and disappears at lift-off. Countdown digits and "TIME'S UP!" are full-screen overlays; the victory message is not an overlay at all — it is written in the sky itself (section 5.5).

3.4 Playing the mission

Purple, gold-veined boulders cannot be moved, they can be shot with the F laser. About half of them hide one to three crystals, which scatter around the blast point, spinning and bobbing; driving over a crystal collects it with a burst of green shards. Ordinary red boulders can be grabbed (P), carried, dropped (O) or thrown (T). Bringing them to the glowing red disc at the foot of the rocket's ramp loads them into the hold: a rock dropped or thrown so that it comes to rest inside the disc disappears into the rocket and the cargo counter ticks up.

The first loaded rock arms the five-minute timer. When the hold is full the three lamps over the hatch and the pad itself turn green; when the crystal goal is also met, the 3-2-1 countdown fills the screen and the ship climbs away on a slow, accelerating arc with a flickering exhaust flame. Ten seconds into the climb the camera, which has eased upward to follow the ship, finds "MISSION ACCOMPLISHED" fading in among the stars as a brand-new constellation. If the timer runs out first, the mission fails and the game reloads to the menu. The R key repositions a stuck rover without erasing progress. Finding the way back after a long crystal hunt is never a guessing game: the rocket compass under the timer always points at the pad and reads out the distance, while the constellations let the Dippers and Polaris serve as a scenic backup.

4. Architecture and code organization

4.1 Structure of the code

`GGame.js` is organized as a sequence of commented, self-contained sections that build the world at load time, renderer and scene, sky, terrain function and ground patch, textures, rock generator, gameplay systems, rover hierarchy, rocket followed by the input handlers and one animation loop that ties everything together. State is kept in a handful of top-level variables (position, heading, velocity, carried rock, jump state, mission counters, launch phase), which keeps the data flow easy to follow: every frame reads input, updates state, and writes it into the scene graph.

4.2 The frame pipeline

Each animation frame, with the frame delta clamped to at most 50 ms so a background tab cannot produce a physics explosion, performs the following steps in order:

1. integrate driving: acceleration, coast friction, steering, position update;
2. resolve collisions against boulders and the rocket platform (circle push-out);
3. recentre the ground patch and refresh the procedurally spawned rocks;
4. advance the jump arc, place the rover on the terrain and slerp it to the slope normal;
5. move the sky and sun anchors to the rover;
6. run the hierarchy animations: head aim, arm sway/stretch, chassis bounce, carried-rock lerp;
7. step the transient systems: flying rocks, laser beams, debris, crystals, pad pulse;
8. advance the mission logic: pickups, launch state machine, timer;
9. update the chase camera and render with the active camera.

5. Technical aspects — world and rendering

5.1 Scene, cameras and fog

The renderer is a WebGL canvas with antialiasing, resized live with the window. Two perspective cameras exist at all times: a chase camera that follows a point behind the rover, smoothed with `lerp` so it swings naturally through turns and jumps, and a first-person “eye” camera parked inside the rover’s head group. Because a Three.js camera is a scene-graph node like any mesh, the eye camera inherits the full hierarchy chain: iit drives with the rover, tilts with the terrain and pans with the head keys, at no extra code cost. The C key simply switches which camera renders. Exponential fog in a dusty-rust tone hides the edge of the finite ground patch and gives the landscape depth; sky elements opt out of fog, since stars are beyond the atmosphere.

5.2 Lights

Four lights illuminate the scene: a warm directional “sun” whose position follows the rover so the light direction is constant everywhere on the infinite plain; a soft ambient fill; and two SpotLights forming the toggleable headlights. Lamp meshes, spotlights and their targets are all children of the chassis, so the light cones follow every bounce and tilt of the vehicle automatically. Emissive materials complete the picture: the beacon, the eye irises, the lamp faces, the crystal glow, the cargo pad and every rocket flame.

5.3 Procedural terrain

The terrain is a function, not a stored mesh: `terrainH(x,z)` returns the ground height at any point, as a sum of sines at three frequencies (dunes) plus two grids of hashed craters: rare bowls the rover can drive down into, and frequent small pockmarks:

```
function terrainH(x,z){ return baseH(x,z) + cratersAt(x, z); }
function baseH(x, z){
  return Math.sin(x*0.021 + 1.7) * Math.cos(z*0.017) * 4.2   // large hills
       + Math.sin(x*0.052) * Math.cos(z*0.047 + 0.8) * 1.6   // medium dunes
       + Math.sin(x*0.19)  * Math.sin(z*0.16) * 0.22;        // small bumps
}
```

Each crater is a smooth bowl with a raised rim, deterministic because its position, radius and depth come from a hash of its grid cell, so the world is identical on every visit and every reload. The visible mesh is a single 300 m plane whose vertices are displaced by the function; when the rover crosses a grid step the patch silently recentres on it, re-displacing vertices at exactly the same world positions, so the world never ends and never crawls. Since the same function drives both the mesh and the vehicle, the rover physically descends into every hole it sees. A per-vertex color attribute bakes darkness into crater bowls: a cheap ambient-occlusion effect that reads better on an open nocturnal landscape than real-time shadows. The terrain normal, obtained by finite differences, tilts the rover to the slope through a quaternion slerp.

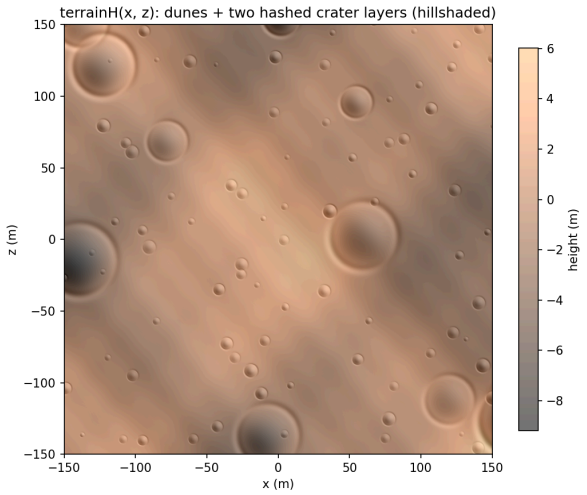


Figure 1 — The terrain height function: dunes from three sine octaves plus two deterministic crater layers, hillshaded.

5.4 Procedural textures — four kinds

All texture maps are drawn at load time on 2D canvases, stamped at wrapped offsets so they tile seamlessly, and used through CanvasTexture. The table lists them; note that the normal map is not painted but computed, exactly as an offline normal-map generator would, from the height canvas:

```
const sx = (hAt(x+1, y) - hAt(x-1, y)) / 255 * STR;   // slope in x
const sy = (hAt(x, y+1) - hAt(x, y-1)) / 255 * STR;   // slope in y
const inv = 1 / Math.sqrt(sx*sx + sy*sy + 1);         // normalize (x, y, 1)
pixel.rgb = [(-sx*inv*0.5 + 0.5), (sy*inv*0.5 + 0.5), (inv*0.5 + 0.5)];
```

Kind	Applied to	How it is generated
Color map	Terrain	Rust base, warm/cool tone patches, thousands of pebble specks, stamped tileably on a 1024² canvas
Normal map	Terrain	Computed in JavaScript from the height canvas by finite differences and encoded as tangent-space RGB
Bump map	All rocks	The grayscale height canvas reused as bump detail (kept on a separate material because Three.js ignores bumpMap when normalMap is present)
Color map	Purple boulders	Dark basalt mottling plus branching gold veins drawn as random walks
Metalness map	Purple boulders	The same vein paths stroked white on black, so metalness = 1 exactly along the veins and glitter specks
Color map (text)	License plate	The “GGame” plate is text drawn on a canvas used as a texture

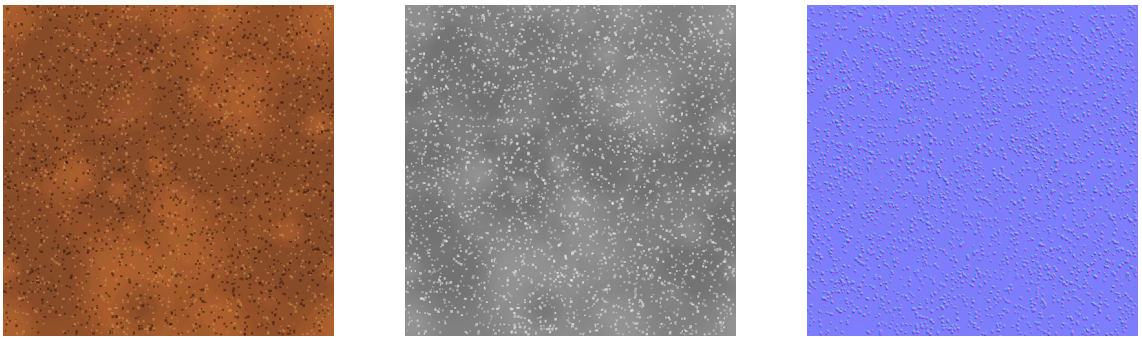


Figure 2 — The ground maps: color (left), height/bump (centre) and the tangent-space normal map computed from it (right). All three tile seamlessly.

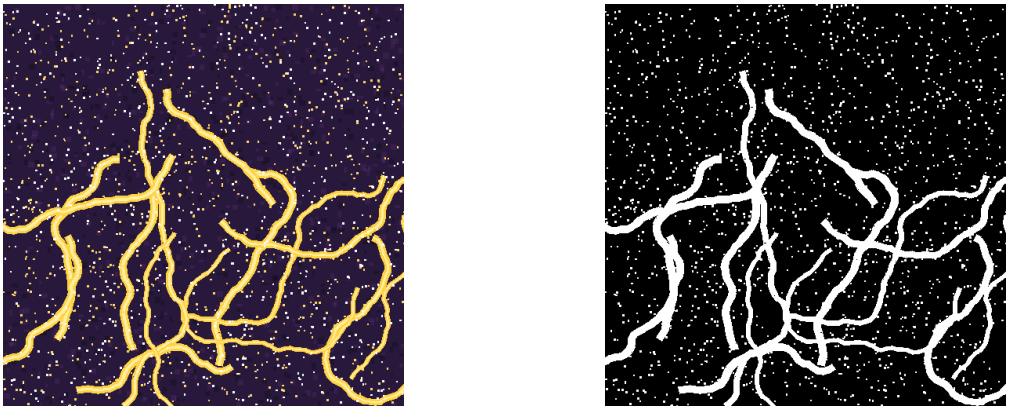


Figure 3 — The purple boulder maps: color with gold random-walk veins and glitter (left) and the matching metalness map (right) — metal = 1 exactly along the veins, so only the gold catches the sun and the headlights.

5.5 Sky

The sky is a shell of 900 random points on the upper hemisphere, with constant pixel size and fog disabled. Ursa Major and Ursa Minor are drawn from small hand-made star charts of azimuth/elevation offsets, joined by faint lines; Polaris sits at the tip of the Little Dipper's handle, bigger, warmer and twinkling through a slow size pulse. The whole sky group follows the rover, so the stars never get closer and always mark north, they work as a real navigation aid when driving back to the rocket. The sky also hosts the finale: when the rocket climbs away, "MISSION ACCOMPLISHED" is written onto the same star shell as a new constellation, centred on the patch of sky the ship is climbing into. Each letter is a small hand-made star chart in the style of the Dippers: big, warm-yellow stars at the letter vertices and a scatter of lesser stars of two magnitudes filling the strokes, placed at uneven steps and nudged slightly off their lines.

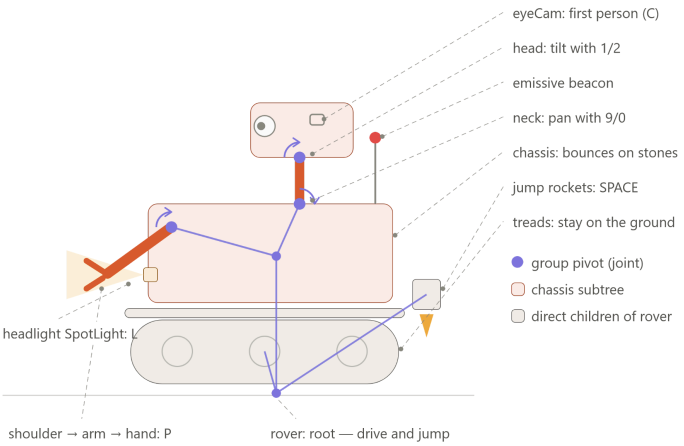
6. Technical aspects — rover, physics and gameplay

6.1 The rover as a hierarchical model

The rover is the required complex hierarchical model. Groups act as joints: each group is placed at the pivot point and the limb is added inside it, shifted away from the pivot, so rotating the group rotates the limb around the joint. The full tree:

```
rover (Group)
  ├─ tread left / tread right (+ fenders, reflective plates)
  ├─ 2 × thruster bell — flame cone          (jump rockets)
  └─ chassis (Group)                         (bounces; treads stay down)
      ├─ body box, front hatch, license plate
```

```
    ├─ 2 × shoulder — arm — hand          (rotate shoulder = arm swings)
    ├─ neckBase — neck
    │             └─ head — 2 × eye pod (lens, iris)
    │                       └─ eyeCam          (first-person camera)
    ├─ 2 × headlight (lamp + SpotLight + target)
    └─ antenna, beacon
```



The animations exploit this structure everywhere. Rotating the head with keys 1/2/9/0 aims both eyes and the first-person view together. The arm sway scales with speed and fades into a forward stretch while grabbing, scaling the shoulder group stretches the entire limb. The chassis bounces and rattles over stones while the treads stay glued to the ground. And a carried boulder is re-parented into the chassis with `attach()`, which preserves its world position, so it follows every bounce and tilt for free until it is dropped back into world space; while carried it is eased into the “hands” position with a simple local-space lerp.

6.2 Driving and collisions

The vehicle model is three numbers: position, heading, signed speed integrated each frame with acceleration, a friction that is strong only while coasting, and speed-scaled steering that inverts in reverse. All moving bodies are circles on the ground plane: big boulders push the rover back to the contact edge and stop it, while small stones are crossable but trigger a fading “bump” jolt that shakes the chassis and steals speed, scaled with the size of the stone. The rocket platform is a solid circle too, while the pad and the ramp are deliberately drive-over.

6.3 Hand-written physics: throws, rolls, jumps, debris

A thrown boulder flies on a ballistic arc under real Mars gravity (3.71 m/s²), then rolls: the terrain gradient accelerates it downhill while **Coulomb friction** — a constant braking deceleration, like real dry friction — brings it to a definitive stop, so a rock thrown into a crater oscillates across the bowl and settles at the bottom:

```
const gx = (terrainH(x+e, z) - terrainH(x-e, z)) / (2*e);   // terrain
gradient
f.vx -= gx * MARS_G * 2.5 * dt;                              // downhill pull
const brake = 0.35 * MARS_G * dt;                            // dry friction
if(speed <= brake){ /* at rest: becomes a normal obstacle */ }
else { f.vx -= f.vx / speed * brake; }                       // brake against
motion
```

Visual spin is applied around the axis perpendicular to the motion at distance/radius, like a real rolling stone. The jump rockets give the rover a vertical ballistic hop stored as an offset above the terrain, so landing on a slope means landing on the slope; a high enough hop

clears boulders entirely, which makes the thrusters tactically useful and not just decorative. Exploded boulders burst into fragments that fly outward, fall, bounce with damping and shrink away; crystal pickups reuse the same debris system with gem-shaped shards.

6.4 Infinite rocks with object pooling

Rocks follow the same deterministic-grid idea as craters: each 16 m cell hashes whether it holds a big boulder (half of them the purple gold-veined variant, built as a grape-cluster of spheres so its silhouette is all lobes) and up to two small stones. Only rocks within about 112 m of the rover exist as meshes, recycled through two pools, one for plain dodecahedra, one for the compound golden clusters, so memory stays constant while driving. A bookkeeping map records every rock the player ever touched, grabbed, re-placed, destroyed or loaded into the rocket so the generator never respawns it at its birthplace.

6.5 Laser and crystals

The laser is a hit-scan: a ray marched from the rover's head until terrain or a boulder stops it, then drawn as a fading green beam with a flash at the impact point. Only purple boulders are destructible; a deterministic hash decides which of them hide crystals, so the player never knows before shooting. Freed crystals are emissive octahedra that spin and bob in place; every frame the pickup routine scans them against the rover position and collects any within reach, revealing the HUD counter on the first one.

6.6 Rocket, cargo and the launch state machine

The rocket is assembled from primitives with its own livery materials (blue hull, red nose, deep-blue fins, white platform, hatch and ramp) and split into two groups: the ground equipment and the lifting "ship" (body, nose, fins, hatch, status lamps, exhaust flame). Loading a rock removes it from the world with the same permanent bookkeeping as an explosion. The mission logic is a four-phase state machine, evaluated after every rock load and crystal pickup, so whichever goal completes last is the one that triggers the countdown:

Phase	Entered when	Behaviour
`0 — idle`	Game start	Normal play. Loading the first rock arms the 5-minute timer; filling the hold turns lamps and pad green.
`1 — countdown`	Both goals complete (either order)	Full-screen 3, 2, 1 digits, then "LIFT-OFF!"; the timer stops and hides.
`2 — climbing`	Countdown expired	Ship altitude = $0.9 \cdot t^2$, slow roll, flame flickering each frame; the camera tilts up to follow the ship; at t = 10 s the "MISSION ACCOMPLISHED" constellation is built into the sky on the ship's azimuth.
`3 — gone`	Altitude > 260 m	Ship hidden; platform, ramp and pad remain; the camera keeps gazing at the constellation; play may continue freely.

6.7 Interface and audio

All 2D interface as menu, difficulty buttons, dashed control cards, counters, timer, rocket compass, countdown digits, the “TIME’S UP!” banner is HTML/CSS layered over the canvas and updated from the game loop; this keeps text crisp at any resolution and cleanly separates presentation from the scene. The one deliberate exception is the victory text, which lives in the 3D scene as a constellation: it belongs to the world, not to the interface. The rocket compass shows how cheap such an overlay can be: each frame the bearing from rover to pad is computed with atan2 in the same convention as the heading, and the difference between the two is applied to the arrow glyph as a plain CSS rotation. The soundtrack is an HTML Audio element created lazily on the first user gesture, as browsers require, and looped.

7. Implemented interactions summary

For the evaluator’s convenience, the interaction examples named in the project brief map to the game as follows. Turning lights on and off: the L key toggles the headlight spotlights. Changing viewpoint: the C key switches between the chase camera and the in-head eye camera, and keys 1/2/9/0 aim the head (and therefore the first-person view). Changing difficulty: the EASY and HARD buttons on the start screen change the crystal and rock goals and rewrite every goal number shown in the interface. Beyond these, the player drives the rover, jumps, grabs, carries, drops and throws boulders, shoots the laser, collects crystals, loads the rocket’s hold, races a timer and toggles the soundtrack, so there are thirteen keys and four menu buttons in total.

8. Performance considerations

The game targets full frame rate on integrated graphics, and several choices serve that goal. Geometry is bounded: one terrain patch whose vertices are re-displaced only when the rover crosses a 2 m grid step, and a rock population capped by the spawn radius and recycled through pools, so the scene never accumulates meshes and the garbage collector is never provoked mid-game. Lighting is cheap by design: no shadow mapping at all, crater darkness is baked into vertex colors once per patch update, and the two spotlights are the only local lights. All textures are generated once at load time; at runtime they cost the same as any static texture, with anisotropic filtering enabled for grazing angles. Transient effects (laser beams, debris, gem shards) live in short arrays with explicit lifetimes and remove themselves, keeping the per-frame update loops tiny.

9. Design choices and known limitations

A few deliberate choices are worth stating so they are not mistaken for defects. The R key resets only the rover’s pose, not the mission: it is a rescue key for awkward positions, while a full restart is a page reload away. Shooting and jumping are disabled while carrying a boulder, the laser would blast the cargo held in front of the head, and the loaded rover is too heavy to lift, which doubles as a gentle difficulty mechanic. The mission timer is armed by the first loaded rock rather than at PLAY, a pacing decision: exploration is unhurried, commitment starts the clock. Crater shading is baked instead of shadow-mapped, a stylistic and performance choice for an open nocturnal landscape. Physics is intentionally bespoke rather than engine-based: the game needs exactly circles, arcs and rolling, and a hand-written version of those is faster, smaller and fully understood. Finally, the music file is taken on Youtube.

10. References

Three.js r128 documentation and examples as threejs.org. Course slides, Interactive Graphics, Prof. M. Schaerf, Sapienza University of Rome. MDN Web Docs for the Canvas 2D API (procedural textures). The crater profile and the normal-map-from-height technique follow the standard formulations used by offline terrain and normal-map generation tools, reimplemented from scratch in `GGame.js`.